**Custom Database**
Deadline: Friday, 21st of November 2025

**You must implement your own programs (absolutely *do not* copy and paste from others) and provide your own answers where appropriate.**

**Note that lacking proper comments will lose mark.**

# 1 Description

In the previous exercises, combinatorial logic was used to build many of the logic circuits that form the foundation of a computer. In this exercise, a simple database with eight registers and basic functionality will be implemented. In addition, arithmetic operations and overflow detection will be included.

| Input pins | | Output pins | |
|---|---|---|---|
| Name | Value | Name | Value |
| bookID[16] | 0 | outID[16] | 0 |
| bookPAGE[16] | 0 | outPAGE[16] | 0 |
| load | 0 | sum[16] | 0 |
| probe | 0 | avg[16] | 0 |
| address[3] | 0 | overflow | 0 |
| sl | 0 | | |
| priority[2] | 0 | | |

Figure 1: book database

# 2 Coursework Tasks

## Task 1: Load and Probe

Turn on **load** (set **load** = 1) to start loading the **bookID** and **bookPAGE** to the database with the assigned **address**.

**Input**:

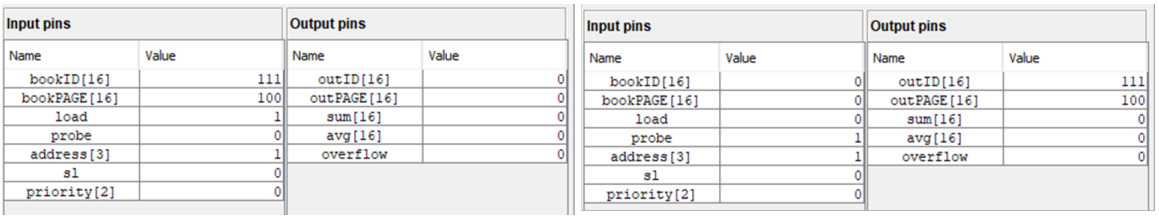
1. A 16-bit version of a book ID named **bookID**.
2. An associated book page quantity named **bookPAGE**.
3. A 1-bit switch for loading the data named **load**.
4. A 3-bit value is assigned to the **address**, which stores each value in 1 of 8 designated registers.
5. A 1-bit switch for probing the data named **probe**.

Turn on **load** (set **load** = 1), then input **bookID** and **bookPAGE**, which will be stored at the specified **address**. For example, when **load** = 1, **bookID** = 111, **bookPAGE** = 100, and **address** = 1. After clicking the **single step** button, the ID and page quantity should be stored in the register.

**Output**: The book ID and page quantity will be output based on the assigned address.

1. A 16-bit version of a book ID named **outID**.
2. An associated book page quantity named **outPAGE**.

Turn on the **probe** (set **probe** = 1) to request data at the specified **address**. For example, after loading the input above, set **probe** = 1 and **address** = 1. The output should display the corresponding ID and page quantity, such as **outID** = 111 and **outPAGE** = 100, which were loaded earlier.



| Input pins | |
|---|---|
| Name | Value |
| bookID[16] | 111 |
| bookPAGE[16] | 100 |
| load | 1 |
| probe | 0 |
| address[3] | 1 |
| sl | 0 |
| priority[2] | 0 |

| Output pins | |
|---|---|
| Name | Value |
| outID[16] | 0 |
| outPAGE[16] | 0 |
| sum[16] | 0 |
| avg[16] | 0 |
| overflow | 0 |

| Input pins | |
|---|---|
| Name | Value |
| bookID[16] | 0 |
| bookPAGE[16] | 0 |
| load | 0 |
| probe | 1 |
| address[3] | 1 |
| sl | 0 |
| priority[2] | 0 |

| Output pins | |
|---|---|
| Name | Value |
| outID[16] | 111 |
| outPAGE[16] | 100 |
| sum[16] | 0 |
| avg[16] | 0 |
| overflow | 0 |

Figure 2: load and probe example

(30 marks)

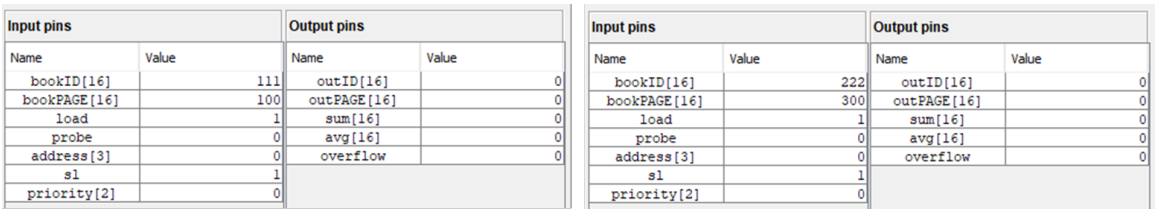
## Task 2: Sequential Load

Based on **Task 1**, implement the sequential load functionality. If the sequential load (**sl**) is turned on, data will be loaded into the RAM in sequential order and will overwrite existing values once all the RAM chips are full. Consequently, this creates a Sequential Access Memory (SAM).

**Input**:

1. A 16-bit version of a book ID named **bookID**.
2. An associated book page quantity named **bookPAGE**.
3. A 1-bit switch for loading the data named **load**.
4. A 1-bit switch for sequential loading the data named **sl**.
5. A 3-bit value is assigned to the **address**, which stores each value in 1 of 8 designated registers.
6. A 1-bit switch for probing the data named **probe**.

For example, in Figure 3: set **load** = 1 and **sl** = 1, first load book with **bookID** = 111 and **bookPAGE** = 100 (stored in the first register by default), then load book with **bookID** = 222 and **bookPAGE** = 300 (stored in the second register for the sequential loading)



| Input pins | | Output pins | |
|---|---|---|---|
| Name | Value | Name | Value |
| bookID[16] | 111 | outID[16] | 0 |
| bookPAGE[16] | 100 | outPAGE[16] | 0 |
| load | 1 | sum[16] | 0 |
| probe | 0 | avg[16] | 0 |
| address[3] | 0 | overflow | 0 |
| sl | 1 | | |
| priority[2] | 0 | | |

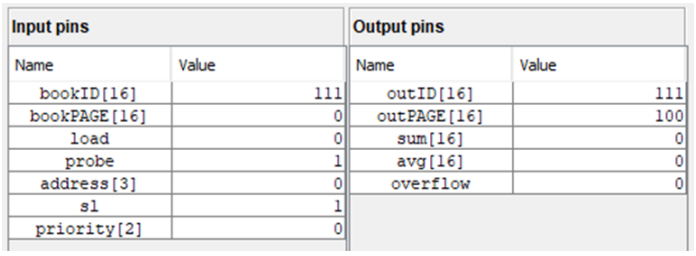
| Input pins | | Output pins | |
|---|---|---|---|
| Name | Value | Name | Value |
| bookID[16] | 222 | outID[16] | 0 |
| bookPAGE[16] | 300 | outPAGE[16] | 0 |
| load | 1 | sum[16] | 0 |
| probe | 0 | avg[16] | 0 |
| address[3] | 0 | overflow | 0 |
| sl | 1 | | |
| priority[2] | 0 | | |

Figure 3: sequential load example

**Output**: The book page quantity will be output based on the assigned book ID.

1. A 16-bit version of a book ID named **outID**.
2. An associated book page quantity named **outPAGE**.

Turn on **probe** (set **probe** = 1) and **sl** (set **sl** = 1) to require the book page quantity with the appointed **bookID**. For example, in Figure 4: set **probe** = 1 and **sl** = 1, with the data already entered above, request the page quantity for the book whose **bookID** = 111. The **outPAGE** show 100 which is related to the same book.



| Input pins | | Output pins | |
|---|---|---|---|
| Name | Value | Name | Value |
| bookID[16] | 111 | outID[16] | 111 |
| bookPAGE[16] | 0 | outPAGE[16] | 100 |
| load | 0 | sum[16] | 0 |
| probe | 1 | avg[16] | 0 |
| address[3] | 0 | overflow | 0 |
| sl | 1 | | |
| priority[2] | 0 | | |

Figure 4: sequential probe example

(25 marks)

## Task 3: Sum of Page Quantities

Take the functionality of the previous task and add the ability to use a 2-bit entry named '**priority**'. When **priority** is set to 1, the chip will calculate the sum of all PAGE values stored in the RAM chips and output the value to sum[16]. If this value is greater than the maximum value for a 16-bit signed integer, then the overflow value should be set to 1.

**Input**:

1. A 16-bit version of a book ID named **bookID**.
2. An associated book page quantity named **bookPAGE**.
3. A 1-bit switch for loading the data named **load**.
4. A 2-bit switch for function priority named **priority**.
5. A 3-bit value is assigned to the **address**, which stores each value in 1 of 8 designated registers.
6. A 1-bit switch for probing the data named **probe**.

**Output**:

1. A 16-bit version of the sum of page quantities named **sum**.
2. A 1-bit overflow flag shows if the sum of page quantity overflows, named **overflow**.

Turn on **probe** (set **probe** = 1) and set **priority** = 1 to get the sum of book page quantities. For example as shown in Figure 5, with the same data loaded in **Task 2**, the **sum** of page quantities (100 + 300) is 400. Since the sum does not cause an overflow, the **overflow** is 0.

| Input pins | | Output pins | |
|---|---|---|---|
| Name | Value | Name | Value |
| bookID[16] | 0 | outID[16] | 0 |
| bookPAGE[16] | 0 | outPAGE[16] | 0 |
| load | 0 | sum[16] | 400 |
| probe | 1 | avg[16] | 0 |
| address[3] | 0 | overflow | 0 |
| sl | 0 | | |
| priority[2] | 1 | | |

Figure 5: sum example

(15 marks)

## Task 4: Simple Average of Page Quantities

Take the functionality of the previous task and add the ability to use a 2-bit entry named '**priority**'. When **priority** is set to 2, the chip will calculate the simple average of all page quantities stored in the RAM chips and output the value to avg[16]. This means that, regardless of the number of books in the database, the average can be obtained simply by dividing the sum of page quantities by 8.

**Input**:

1. A 16-bit version of a book ID named **bookID**.
2. An associated book page quantity named **bookPAGE**.
3. A 1-bit switch for loading the data named **load**.
4. A 2-bit switch for function priority named **priority**.
5. A 3-bit value is assigned to the **address**, which stores each value in 1 of 8 designated registers.
6. A 1-bit switch for probing the data named **probe**.

**Output**:

1. A 16-bit version of the average of page quantities named **avg**.

Turn on **probe** (set **probe** = 1) and set **priority** = 2 to get the simple average of book page quantities. For example, as shown in Figure 2, with the same data loaded in **Task 2**, the simple average of page quantities, **avg**, is calculated by (100 + 300) / 8 = 50.

| Input pins | | Output pins | |
|---|---|---|---|
| Name | Value | Name | Value |
| bookID[16] | 0 | outID[16] | 0 |
| bookPAGE[16] | 0 | outPAGE[16] | 0 |
| load | 1 | sum[16] | 0 |
| probe | 0 | avg[16] | 50 |
| address[3] | 0 | overflow | 0 |
| sl | 0 | | |
| priority[2] | 2 | | |

Figure 6: simple average example

(10 marks)

## Task 5: Average of Page Quantities

Take the functionality of the previous task and add the ability to use a 2-bit entry named '**priority**'. When **priority** is set to 3, the chip will calculate the actual average of all book page quantities stored in the RAM and output the result to avg[16]. To calculate the actual average page count, the number of entries in the database must be considered, excluding those with a **bookID** of 0. The average of page quantities should be rounded down to the nearest whole number.

**Input**:

1. A 16-bit version of a book ID named **bookID**.
2. An associated book page quantity named **bookPAGE**.
3. A 1-bit switch for loading the data named **load**.
4. A 2-bit switch for function priority named **priority**.
5. A 3-bit value is assigned to the **address**, which stores each value in 1 of 8 designated registers.
6. A 1-bit switch for probing the data named **probe**.

**Output**:

1. A 16-bit version of the average of page quantities named **avg**.

Turn on **probe** (set **probe** = 1) and set **priority** = 3 to get the average of book page quantity. For example, as shown in Figure 7, with the same data loaded in **Task 2**, the actual average of page quantities, **avg**, is calculated by (100 + 300) / 2 = 200.

| Input pins | | Output pins | |
|---|---|---|---|
| Name | Value | Name | Value |
| bookID[16] | 0 | outID[16] | 0 |
| bookPAGE[16] | 0 | outPAGE[16] | 0 |
| load | 0 | sum[16] | 0 |
| probe | 1 | avg[16] | 200 |
| address[3] | 0 | overflow | 0 |
| sl | 0 | | |
| priority[2] | 3 | | |

Figure 7: actual average example

(10 marks)

# 3 Submission Requirements

Please place all your developed files into a folder named 'Coursework1 Assignment', compress this folder into a .zip file, and name the archive exactly as follows (all lowercase, no spaces, no extra characters): '**yourstudentid_yourname.zip**'.

The zip archive must contain:

1. CW.hdl – the main HDL file.
2. Every additional HDL file referenced by CW.hdl must also be included.
3. A `readme.txt` containing usage notes (optional).

Upload the zip file to the Moodle submission box. Each new upload completely replaces the previous one, so if you resubmit, ensure that the final package includes all required files, including chips, test scripts, and documentation. Submissions that violate the naming or formatting rules—even by a small margin—will receive 0 marks. Late submissions incur the standard penalty: 5% of the maximum mark for every 24-hour period (or part thereof) past the deadline.

# 4 Marking Criteria

## 4.1 Evaluation Form

Table 1: Evaluation Form – Total 100%

| **Task** | **Weight** |
|---|---|
| Task 1 | 30% |
| Task 2 | 25% |
| Task 3 | 15% |
| Task 4 | 10% |
| Task 5 | 10% |
| Appropriate comments | 10% |

## 4.2 Test Specifications

- The `.cmp` files provided in `CW1_skeleton.zip` are *illustrative only*.
- The actual test set is *more comprehensive*, covering *overflow*, *boundary values*, and *unspecified input patterns*.
- To achieve *full marks*, your design must correctly handle *all* valid input cases, including those not explicitly mentioned (You can describe the design in comments or readme).
- Each task is evaluated using *multiple hidden test cases*. Your score for each task is proportional to the *percentage of test cases passed*.

## 4.3 Boundary Case Examples

- **Task 1** performs *partial loading* based on the address. In **Task 2**, when loading sequentially, addresses already written in **Task 1** must be *skipped*, and *sequential overwrite* should begin only after all registers are used.
- All **bookPAGE** values in the provided .tst file are *non-zero*.
- For **sum-related operations**, *overflow* must be handled. Since overflow may occur *multiple times*, the **overflow flag** must be set to **1** *if any overflow occurs*.

## 4.4 Important Notes:

- Incorrect file name, format, or unreadable files → **0 marks** for the entire coursework.
- Each submission overwrites the previous one; only the *final upload* will be graded.
- **Late submission penalty:** 5% of the total marks will be deducted for every 24-hour period (or part thereof) past the deadline.